

CICD 过程文档

1. GitLab Runner 搭建与配置过程

1.1 Runner 所在机器的环境配置

Runner 所在机器安装 Docker，能正常执行 `docker build` 和 `docker run`。由于机器无法直接访问外网，需要配置 Docker 国内镜像源，保证后续构建和拉取依赖可正常进行。

1.2 安装 Runner

```
# 下载二进制文件
sudo curl -L --output /usr/local/bin/gitlab-runner "https://gitlab-runner-downloads.s3.amazonaws.com/latest/binaries/gitlab-runner-linux-amd64"

sudo chmod +x /usr/local/bin/gitlab-runner
# 创建用户并安装服务
sudo useradd --comment 'GitLab Runner' --create-home gitlab-runner --shell /bin/bash
sudo gitlab-runner install --user=gitlab-runner --working-directory=/home/gitlab-runner
sudo gitlab-runner start
```

1.3 注册 Runner

运行 gitlab 上提示的注册命令，将机器与 GitLab 关联

1.4 配置 Runner 权限与执行环境

使 Runner 具备 Docker 权限，并能读写工作目录

1.5 Runner 最终效果

A Agent_Evaluation_System...

项目信息

仓库

议题0

合并请求0

CI/CD

安全与合规

部署

软件包与镜像库

基础设施

监控

分析

Wiki

代码片段

设置

2026seiii-072-V我50万Token > Agent_Evaluation_System_front > CI/CD 设置 > #470 (K1AmA4SJK)

Runner #470

项目

属性名称	值
启用	是
受保护	否
可以运行未打标签的作业	是
锁定到此项目	否
标签	nju-runner shell
名称	
版本	18.3.1
IP地址	172.29.4.218
版本	5a021a1c
平台	linux
架构	amd64
描述	
最大作业超时	
最后联系	38分钟前

2. 项目范围

本项目包含三个独立仓库：

仓库	主要技术栈	当前 CI 文件	当前构建结果
agent	Python / Agent 推理核心	agent/.gitlab-ci.yml	Docker 基础镜像 + 单元测试报告
agent_evaluation_system_front	Vue 3 + Vite 前端	agent_evaluation_system_front/.gitlab-ci.yml	前端 Docker 镜像 + 容器部署
agent-eval-backend	Python / FastAPI 后端	agent-eval-backend/.gitlab-ci.yml	后端 Docker 镜像 + 容器部署

3. 流水线总览

3.1 agent 仓库

阶段为 build、test。流水线会先构建基础镜像，再跑单元测试，测试结果输出为 JUnit 报告。

3.2 前端仓库

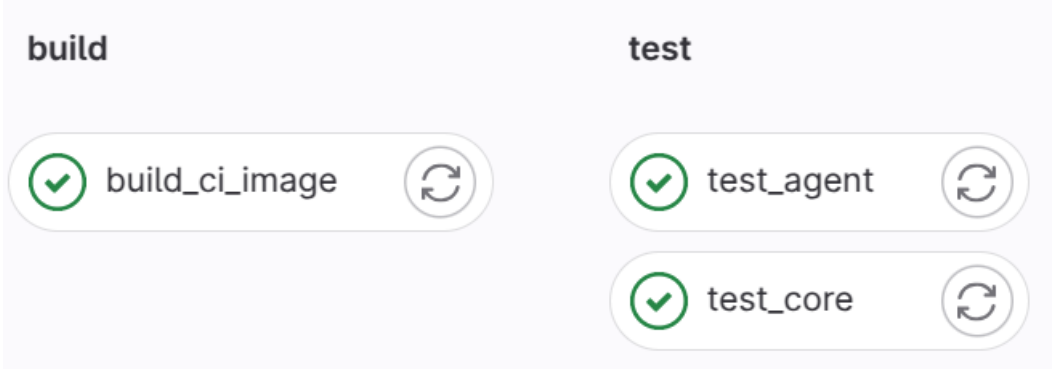
阶段为 build、test、deploy。默认分支触发，先构建前端镜像，再做简单检查，最后在 Runner 机器上更新容器。

3.3 后端仓库

阶段为 check、build、deploy。分支或 tag 触发，先做 smoke test，再构建镜像，最后启动后端容器。

4. agent 仓库 CI 说明

4.1 流水线结构



agent/.gitlab-ci.yml 有 build 和 test 两个阶段。build_ci_image 先生成镜像，test_core 和 test_agent 分别跑核心测试和 Agent 测试。

4.2 build 阶段

任务: `build_ci_image`

- 根据 `requirements.txt` 生成镜像标签。
- 优先复用本地缓存后构建 Docker 镜像。

4.3 test 阶段

任务一: `test_core`

- 运行 `tests/test_preprocess.py` 、 `tests/test_planner.py` 、 `tests/test_scanner.py`。
- 产物: `report-core.xml`。

任务二: `test_agent`

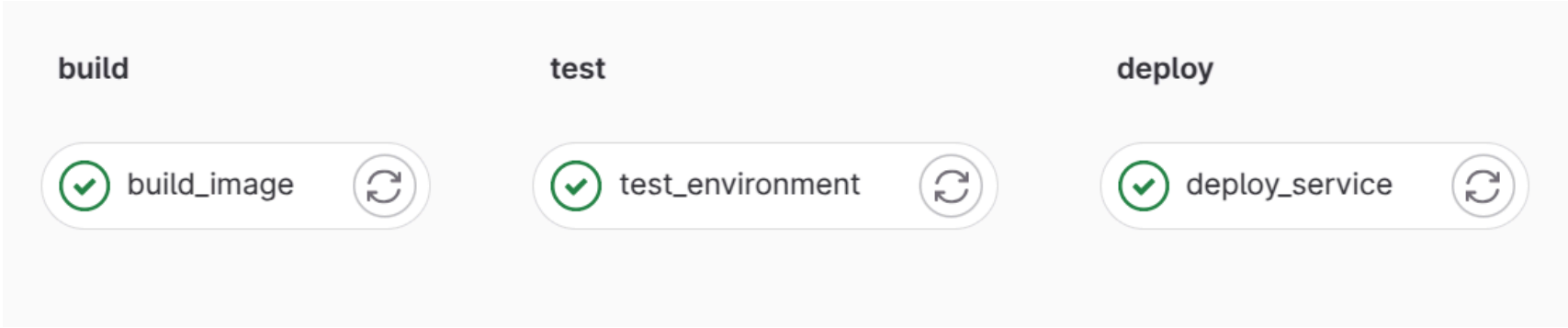
- 运行 `tests/test_agent.py`、 `tests/test_cross_reference.py`。
- 产物: `report-agent.xml`。

4.4 产物

- JUnit 报告: `report-core.xml`、 `report-agent.xml`
- 保留时间: `1 week`

5. 前端仓库 CI/CD 说明

5.1 流水线结构



`agent_evaluation_system_front/.gitlab-ci.yml` 有 `build`、`test`、`deploy` 三个阶段。

5.2 build 阶段

任务: `build_image`

- 在默认分支构建前端 Docker 镜像。
- 镜像名: `agent-frontend:latest`。

5.3 test 阶段

任务: `test_environment`

- 当前是简单健康检查，脚本里主要执行 `node -v`。

5.4 deploy 阶段

任务: `deploy_service`

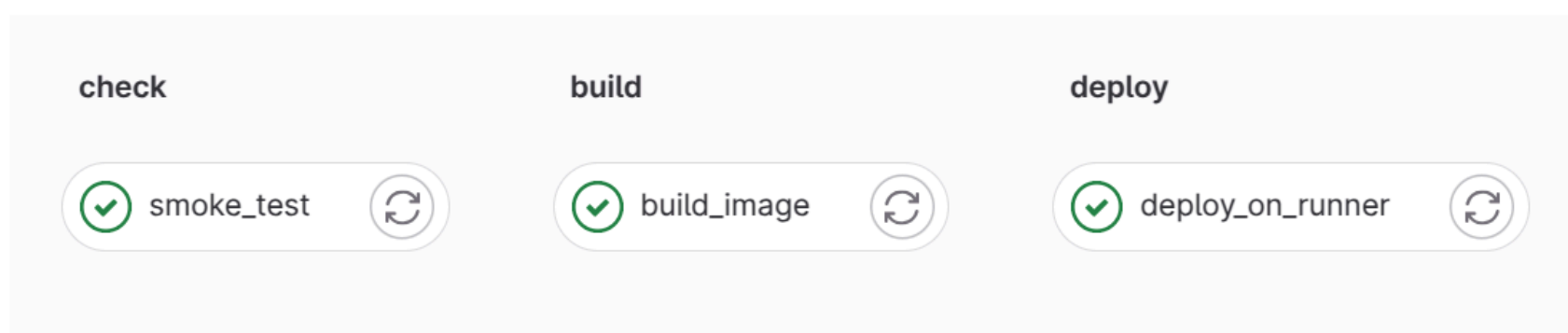
- 停止旧容器后启动新容器。
- 端口映射为 `80:80`。
- 最后清理无用镜像。
- 部署地址: <http://172.29.4.218>

5.5 前端 Docker 构建方式

前端 Docker 构建使用 `node:22-alpine` 完成打包，再用 `nginx:alpine` 提供静态资源，最终暴露 `80` 端口。

6. 后端仓库 CI/CD 说明

6.1 流水线结构



`agent-eval-backend/.gitlab-ci.yml` 有 `check`、`build`、`deploy` 三个阶段。

6.2 统一镜像变量

脚本通过 `docker_setup` 统一导出 `IMAGE_TAG=agent-eval-backend:$CI_COMMIT_SHA`。

6.3 check 阶段

任务: `smoke_test`

- 先构建镜像，再做一个简单启动检查。
- 当前校验是导入 `app.main` 并检查 `app.title`。

6.4 build 阶段

任务: `build_image`

- 直接构建后端镜像。

6.5 deploy 阶段

任务: `deploy_on_runner`

- 删除旧容器后启动新容器，端口映射为 `8000:8000`，并设置自动重启。
- 部署地址: <http://172.29.4.218:8000/>

6.6 后端 Docker 构建方式

后端使用 `python:3.12-slim` 和 `uv` 构建，服务启动端口为 `8000`。